UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 /20242025

PROJECT REPORT

(SECR2043 OPERATING SYSTEMS)

TOPIC:

VIRTUAL MEMORY MANAGEMENT - FIFO AND LRU REPLACEMENT ALGORITHMS

LECTURER NAME:

DR. MUHAMMAD ZAFRAN BIN MUHAMMAD ZALY SHAH

GROUP NAME:

x64 (SECTION 02)

NAME	MATRIC NUMBER
IMAN ABADI BIN MOHD NIZWAN	A23CS0084
MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
MUHAMMAD AMMAR BIN MOHAMAD IDHAM	A23CS0247

Table of Content

Simulating Locker Organization with FIFO and LRU: A Real-World Analogy for Virtual Memory Management

1.0 Abstract

This report explores the concept of virtual memory management in operating systems through a relatable real-world analogy, which is students organising textbooks in lockers to reduce the burden of their heavy school bags. Specifically, it demonstrates how two page replacement algorithms, First-In-First-Out (FIFO) and Least Recently Used (LRU) can be applied to determine which books to keep or remove when locker space is limited. FIFO removes the oldest stored item regardless of how recently it was used. In contrast, LRU prioritises keeping recently accessed items and removes those that haven't been used for the longest time. A Python program was used to simulate a book access sequence and compare the number of evictions under each algorithm. The findings highlight the practical efficiency of LRU in maintaining frequently used items while minimizing unnecessary replacements. This report offers a better way to understand how operating systems optimize limited memory space through page replacement algorithms by relating it to a common student experience.

2.0 Introduction

In operating systems, efficient memory management is important for systems to run as smoothly as possible as well as to allow more programs to run at the same time. An important concept that could achieve this goal is virtual memory, which is the separation of user logical memory from physical memory. According to Silberschatz, Galvin, and Gagne (2013), it allows for only a portion of the program to be in memory for execution. Within this system, demand paging plays a key role by loading pages into memory only when needed. When the system attempts to access a page that is currently not in memory, a page fault occurs. This prompts the operating system to retrieve the page from secondary memory. If physical memory is full, the operating system has to decide which page to remove. This is determined by page replacement algorithms. This report introduces the First-In-First-Out (FIFO) and Least Recently Used (LRU) algorithms through a real-world analogy of students using lockers to store textbooks. This more relatable context would hopefully allow for a better understanding of these concepts, helping to illustrate how operating systems manage limited memory space in practical terms.

3.0 Problem Statement

Nowadays, students are required to carry a lot of books and school materials, which, along with the growing academic standards, proves to be an additional obstacle in the context of elementary education. More than 60% of elementary school pupils, according to recent studies, carry bags that are more than 15% of their body weight (Dockrell et al., 2023; Malhotra et al., 2021, as cited in Pardede & Sobandi, 2024). This can have detrimental effects on not only physical health but also the students' psychological well-being and social development.

4.0 Related Works

Implementing the replacement algorithm on this issue has limited studies. Based on the studies that discuss the effects of elementary students carrying bags, there are recommendations that could address this problem, which are providing designated lockers for each student and having systems that implement textbook rotation (Pardede & Sobandi, 2024). Another study suggests that packing the books in the bag according to the timetable is another viable solution (Saini et al., 2023). From these studies, there is a potential of implementing the replacement algorithm, comparing the result using LRU and FIFO to determine the best algorithm to use to determine which books need to be removed from the locker that has a limited capacity based on the timetable, where, in the context of virtual memory management, the input string. This way, the student does not need to bring back all of the books with them.

5.0 Methodology

To address the issue of heavy school bags among elementary students, this project simulates locker management using two page replacement algorithms which is Least Recently Used (LRU) and First-In, First-Out (FIFO). These algorithms help decide which books to keep in the locker and which to remove when space is limited. LRU works by removing the book that has not been used for the longest time, ensuring frequently used books remain accessible, while FIFO removes the oldest added book regardless of recent usage. By running a step-by-step simulation with a realistic book access sequence, it will demonstrate how these algorithms can minimize the number of books students carry daily.

6.0 Implementation

6.1 Flow Chart

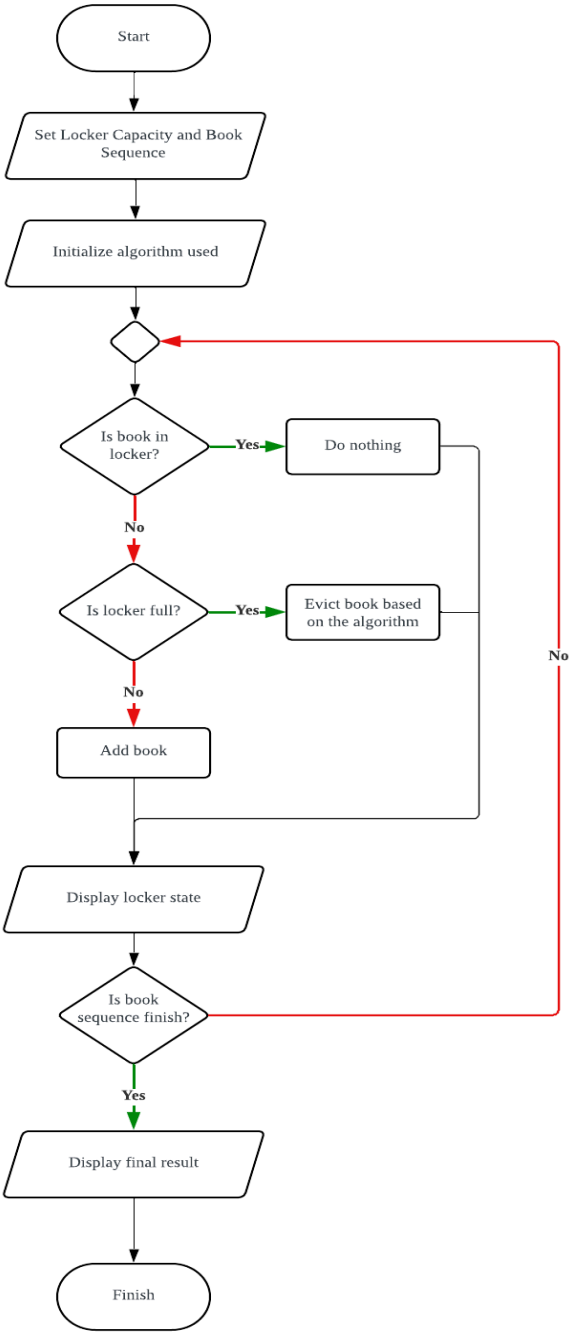

Figure 6.1 Flowchart of Simulation Flow

6.2 Code Implementation

FIFO algorithm:

```
class LockerFIFO:
    Trae: Explain | Doc | Test | Explore IDE | ×
    def __init__(self, capacity):
        self.capacity = capacity
        self.books = deque()
        self.book_set = set()
        self.evictions = 0

    Trae: Explain | Doc | Test | Explore IDE | ×
    def access(self, book):
        if book in self.book_set:
            return  # Already in locker, do nothing
        if len(self.books) >= self.capacity:
            removed = self.books.popleft()
            self.book_set.remove(removed)
            self.evictions += 1
        self.books.append(book)
        self.book_set.add(book)

    Trae: Explain | Doc | Test | Explore IDE | ×
    def state(self):
        return list(self.books)
```

Figure 6.2 LockerFIFO Class Definition

The LockerFIFO class, on the other hand, simulates a locker that evicts books based on the order they entered, regardless of how recently they were accessed. During its **initialization**, it also sets a capacity, uses a deque named books to maintain the order of entry, and employs a set named book_set for efficient checking of a book's presence. An eviction counter is also initialized. When the **access method** is invoked for a book, it first verifies if the book is already present in the book_set; if so, no action is taken as accessing an existing book does not change its position in a FIFO system. If the book is new and the locker has reached its capacity, the access method removes the book that has been in the locker the longest (by calling popleft() on the deque) and removes it from the book_set, incrementing the evictions count. Afterward, the new book is added to the end of both the deque and the book_set. The **state method** provides a list of the books currently in the locker, maintaining their order from the oldest (first-in) to the newest (last-in).

LRU algorithm:

```
class LockerLRU:
    Trae: Explain | Doc | Test | Explore IDE | ×
    def __init__(self, capacity):
        self.capacity = capacity
        self.books = OrderedDict()
        self.evictions = 0

    Trae: Explain | Doc | Test | Explore IDE | ×
    def access(self, book):
        if book in self.books:
            self.books.move_to_end(book)
        else:
            if len(self.books) >= self.capacity:
                self.books.popitem(last=False)
                self.evictions += 1
            self.books[book] = True

    Trae: Explain | Doc | Test | Explore IDE | ×
    def state(self):
        return list(self.books.keys())
```

Figure 6.3 LockerLRU Class Definition

The LockerLRU class represents a locker that prioritizes keeping the books that have been accessed most recently. Upon **initialization**, it sets a maximum capacity for the locker, creates an OrderedDict named books to store the books while maintaining their access order, and initializes an evictions counter. When the **access method** is called with a book, the code first checks if the book is already in the books collection. If it is, that book is moved to the end of the OrderedDict to mark it as the most recently accessed. If the book is not in the locker, the code then checks if the locker has reached its capacity; if it has, the book that has been in the locker the longest without being accessed (the one at the very beginning of the OrderedDict) is removed, and the evictions count is incremented. Finally, the new book is added to the books collection, automatically placing it at the end as the most recently accessed. The **state method** simply returns a list of the books currently in the locker, ordered from least recently used to most recently used.

Simulation Execution:

Configure the main simulation parameter, which is the capacity. Initialize the simulation environment and run the setup process. Finally, execute the simulation.

```
if __name__ == "__main__":
    capacity = 3  # Keep this small to trigger replacements
    sequence = ["Music", "Math", "English", "Science", "Math", "History", "Math", "Art", "Science", "History"

    print("Book Access Sequence (Length = {}):".format(len(sequence)))
    print(sequence)
    print("-" * 80)

    lru_locker = LockerLRU(capacity)
    fifo_locker = LockerFIFO(capacity)
```

Figure 6.4 Simulation Execution Code Segment

Iterating Through The Sequence

Use a for loop to go through the input string in the variable “sequence.” For each sequence, a copy of the locker state and number of evictions is made to store the number of previous evictions before the access method is called. Then a comparison between the current eviction and previous eviction is made to see if an eviction occurred.

```python
for step, book in enumerate(sequence, 1):
    print(f"\nStep {step}: Accessing '{book}'")

    # Store previous states
    lru_prev = lru_locker.state().copy()
    fifo_prev = fifo_locker.state().copy()
    lru_prev_evictions = lru_locker.evictions
    fifo_prev_evictions = fifo_locker.evictions

    # Process the access
    lru_locker.access(book)
    fifo_locker.access(book)

    # Show the results
    print(f"  LRU Before:  {lru_prev}")
    print(f"  LRU After:   {lru_locker.state()}")
    if lru_locker.evictions > lru_prev_evictions:
        print(f"  LRU Action:  EVICTION occurred (Total evictions: {lru_locker.evictions})")
    elif book in lru_prev:
        print(f"  LRU Action:  HIT - moved '{book}' to end")
    else:
        print(f"  LRU Action:  MISS - added '{book}'")

    print(f"  FIFO Before: {fifo_prev}")
    print(f"  FIFO After:  {fifo_locker.state()}")
    if fifo_locker.evictions > fifo_prev_evictions:
        print(f"  FIFO Action: EVICTION occurred (Total evictions: {fifo_locker.evictions})")
    elif book in fifo_prev:
        print(f"  FIFO Action: HIT - no change")
    else:
        print(f"  FIFO Action: MISS - added '{book}'")
```

Figure 6.5 Sequence Iteration Segment

Output Analysis:

This code segment is used to make a final comparison between the LRU and the FIFO algorithm, comparing the number of evictions done by both al.

```
print("\nFINAL COMPARISON RESULTS:")
print("=" * 80)
print(f"LRU Locker Contents:  {lru_locker.state()} | Total Evictions: {lru_locker.evictions}")
print(f"FIFO Locker Contents: {fifo_locker.state()} | Total Evictions: {fifo_locker.evictions}")

# Performance analysis
print(f"\nPerformance Analysis:")
print(f"LRU had {lru_locker.evictions} evictions")
print(f"FIFO had {fifo_locker.evictions} evictions")
if lru_locker.evictions < fifo_locker.evictions:
    print("LRU performed better (fewer evictions)")
elif fifo_locker.evictions < lru_locker.evictions:
    print("FIFO performed better (fewer evictions)")
else:
    print("Both algorithms performed equally")
```

Figure 6.6 Final Output

7.0 Discussion

Example Output:

```
Book Access Sequence (Length = 20):
['Music', 'Math', 'English', 'Science', 'Math', 'History', 'Math', 'Art', 'Science', 'History', 'Math', 'History', 'Science',
 'Science', 'Math', 'English', 'Music', 'Math', 'English']
--------------------------------------------------------------------------------

STEP-BY-STEP EXECUTION:
================================================================================

Step 1: Accessing 'Music'
  LRU Before:  []
  LRU After:   ['Music']
  LRU Action:  MISS - added 'Music'
  FIFO Before: []
  FIFO After:  ['Music']
  FIFO Action: MISS - added 'Music'
----------------------------------------

Step 2: Accessing 'Math'
  LRU Before:  ['Music']
  LRU After:   ['Music', 'Math']
  LRU Action:  MISS - added 'Math'
  FIFO Before: ['Music']
  FIFO After:  ['Music', 'Math']
  FIFO Action: MISS - added 'Math'
----------------------------------------
```

Figure 7.1 Initialization and Simulation Execution Output

```
Step 20: Accessing 'English'
  LRU Before:  ['English', 'Music', 'Math']
  LRU After:   ['Music', 'Math', 'English']
  LRU Action:  HIT - moved 'English' to end
  FIFO Before: ['Science', 'Music', 'Math']
  FIFO After:  ['Music', 'Math', 'English']
  FIFO Action: EVICTION occurred (Total evictions: 12)
----------------------------------------

FINAL COMPARISON RESULTS:
================================================================================
LRU Locker Contents:  ['Music', 'Math', 'English'] | Total Evictions: 9
FIFO Locker Contents: ['Music', 'Math', 'English'] | Total Evictions: 12

Performance Analysis:
LRU had 9 evictions
FIFO had 12 evictions
LRU performed better (fewer evictions)
```

Figure 7.2 Final Step and Result Comparison Output

When it comes to baggage, the LRU (Least Recently Used) method states that students should prioritize carrying resources that they have used most recently or that they expect to require in the future. If the bag is full and a new book is needed, the LRU technique recommends removing the book that hasn't been used in the longest time. This is consistent with the LRU code's approach of moving accessed items to the "most recent" end and removing them from the "least recent" end. This method is particularly effective in situations when the use of material demonstrates "locality of reference" (GeeksforGeeks, 2024). For a school bag, this implies a student would ideally carry only the books for current lessons and homework, resulting in a substantially lighter load and reducing instances of forgetting items.

In contrast, the FIFO (First-In, First-Out) strategy would have students remove materials based on the order they were packed, regardless of their current relevance. A book packed on Monday, for instance, might be the first to be left behind on Wednesday if the bag becomes full, even if it's needed for a class later that same day. The FIFO code's use of a deque and popleft() accurately reflects this chronological eviction. As the simulation often demonstrates, this method can be less efficient than LRU for typical school material access patterns, potentially leading to more "evictions" of still-needed items and, consequently, more instances of forgotten materials or the student having to repeatedly transport items to and from school.

The simulation's eviction counter provides a clear indicator of how effective each packing technique is. A lesser number of evictions indicates that the selected approach is more effective in keeping essentials within the constrained space, similar to a student packing their luggage with only the essentials. The simulation's comparison of the LRU and FIFO lockers' total evictions clearly demonstrates the LRU approach's strategic advantage in minimizing needless baggage load and optimizing the availability of necessary materials, which promotes a more orderly school day for elementary students.

8.0 Future work & Conclusion

8.1 Future Work

The current project demonstrates the potential of implement First-In-First-Out (FIFO) and Least Recently Used (LRU) replacement algorithm for locker organization simulation, but there are plenty of approach for future works to improve the system further:

1. **Incorporate Time-Based Decay:** Enhance the simulation by applying a time-based weighting. The reason is that books that have not been accessed for a very long time are heavily penalized by the system. This will make the system smarter to decide when to throw out the older books.
2. **Statistical Tracking and Visualization:** Improvise the system by using a log and visualize access patterns. For instance, the system can display how many times that student has access to each book.
3. **Interactive User Input or Real-Time Decisions:** This system can be improved by allowing users to freely interact with books on what to access next in real time.
4. **Machine Learning for Prediction:** As a more advanced direction, machine learning can help to integrate a simple prediction to guess which book is more likely to be used next, based on the previous action patterns. This mimics prefetching in modern OS memory management.

8.2 Conclusion

This project has demonstrated a simulation of locker organization using FIFO (First-In, First-Out) and LRU (Least Recently Used) algorithms has provided valuable insight into how different memory management strategies affect resource utilization in space-limited environments. FIFO demonstrated a straightforward approach, evicting the oldest item regardless of its usage, which led to more frequent and sometimes inefficient removals. In contrast, LRU proved to be more effective by retaining recently accessed books and removing only those that had not been used for the longest time, resulting in fewer evictions and better locker usage. By translating these abstract computer memory concepts into a real-world scenario such as managing school locker space, the project successfully highlighted the strengths and limitations of each algorithm in a way that is accessible to non-technical audiences. This practical comparison reinforces the importance of intelligent replacement policies in optimizing limited storage resources, both in computing systems and everyday life.

9.0 Acknowledgement

We would like to extend our heartfelt gratitude to our lecturers, Dr. Muhammad Zafran Bin Muhammad Zaly Shah and Dr. Danlami Gabi, for their dedicated teaching and invaluable support throughout the Operating System course this semester. Their passion for the subject and commitment to student learning have been truly inspiring and have significantly enriched our academic experience.

Throughout the course, both lecturers provided us with a strong foundation in key operating system concepts such as memory management, process scheduling, page replacement algorithms, and others. Their clear and structured explanations, along with practical examples and consistent guidance, have enabled us to better understand the real-world applications of these topics. This also includes their dedication to consistently support us outside lectures like giving some guidance and helpful clues for the upcoming lab exercises, quizzes, and also paper tests.

This project, in particular, has been a meaningful opportunity for us to apply what we have learned in class to simulate and compare the FIFO and LRU replacement algorithms in a relatable context. We are especially thankful for the encouragement and feedback provided during our project development, which helped us refine our ideas and improve the overall quality of our work.

Once again, we sincerely thank both Dr. Muhammad Zafran Bin Muhammad Zaly Shah and Dr. Danlami Gabi for their unwavering support, patience, and dedication throughout the semester. This project would not have been possible without their guidance.

10.0 References

GeeksforGeeks. (2024, September 17). *Locality of reference and cache operation in cache memory*. GeeksforGeeks. https://www.geeksforgeeks.org/locality-of-reference-and-cache-operation-in-cache-memory/

Pardede, M., & Sobandi, A. (2024). The Impact of Carrying Heavy Bags on Elementary School Students: Challenges and Solutions in the Digital Era. Didaktika: Jurnal Kependidikan, 13(001 Des), 1187-1196.

https://mail.jurnaldidaktika.org/contents/article/download/1439/860

Saini, S. K., Bharti, B., & Gopinathan, N. R. (2023). Impact of Educational Intervention in Reducing the Weight of School Bags in Selected School Children. Indian Journal of Community Medicine, 48(4), 533-538. https://journals.lww.com/ijcm/_layouts/15/oaks.journals/downloadpdf.aspx?an=00659070-202348040-00006

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating system concepts* (9th ed.). John Wiley & Sons.

VIDEO LINK :

https://youtu.be/LP1bTE3nI98